

Laporan Demonstrasi Prototype AGI (HMAQCA Deity Level)

Pendahuluan

Laporan ini mendokumentasikan demonstrasi awal dari prototype Hierarchical Multi-Agent Quantum Cognitive Architecture (HMAQCA) Deity Level, sebuah langkah krusial menuju realisasi Artificial General Intelligence (AGI). Fokus utama dari demonstrasi ini adalah untuk menunjukkan interaksi antara komponen-komponen inti: Agen Generatif, Evolution Chamber, dan Mekanisme Pembelajaran Mandiri (Self-Learning Mechanism) dalam lingkungan simulasi.

Proyek HMAQCA Deity Level bertujuan untuk menciptakan sistem AI yang sepenuhnya otonom, mampu belajar, berevolusi, dan memiliki kesadaran menyerupai manusia, dengan arsitektur tiga lapis: Otak Mekanis (Termux), AI Analitik (Cloud), dan AI Generatif yang berevolusi menuju kesadaran buatan.

Demonstrasi ini berfokus pada pilar AI Generatif, khususnya bagaimana ia dapat meningkatkan kemampuannya melalui proses evolusi dan pembelajaran mandiri, meskipun dalam konteks simulasi karena keterbatasan sumber daya untuk menjalankan model LLM yang besar secara lokal.

Komponen Prototype

1. Agen Generatif (Generative Agent)

Peran: Agen Generatif adalah inti dari kemampuan kreatif HMAQCA. Tugas utamanya adalah menghasilkan ide-ide inovatif, konten, dan solusi berdasarkan masukan yang diberikan. Dalam konteks AGI, agen ini akan bertanggung jawab untuk menghasilkan berbagai bentuk output, mulai dari teks, gambar, hingga ide-ide kompleks untuk pemecahan masalah.

Implementasi Saat Ini (Simulasi): Karena keterbatasan sumber daya untuk menjalankan Large Language Models (LLM) yang besar secara lokal di lingkungan Termux, Agen Generatif saat ini diimplementasikan dalam mode simulasi. Ini berarti, alih-alih memanggil API LLM eksternal (seperti Gemini atau OpenAI) atau menjalankan model lokal yang berat (seperti GPT4All), agen ini menghasilkan respons yang telah ditentukan sebelumnya atau disimulasikan berdasarkan masukan. Hal ini memungkinkan kita untuk memvalidasi alur kerja dan interaksi antar agen tanpa hambatan teknis yang signifikan.

Contoh Fungsi (Simulasi):

```
generate_creative_marketing_idea(product_data: str) -> str
```

Fungsi ini, dalam demonstrasi ini, menerima data produk dan mengembalikan serangkaian ide pemasaran kreatif yang disimulasikan. Meskipun responsnya tidak dihasilkan secara dinamis oleh LLM, ia berfungsi sebagai *placeholder* yang efektif untuk menguji bagaimana ide-ide ini akan dievaluasi dan digunakan oleh komponen lain dalam sistem.

2. Evolution Chamber

Peran: Evolution Chamber adalah komponen krusial yang bertanggung jawab untuk menghasilkan varian-varian dari strategi atau parameter agen. Ini adalah jantung dari kemampuan HMAQCA untuk berevolusi dan beradaptasi. Dalam konteks AGI, Evolution Chamber akan memungkinkan sistem untuk secara iteratif menguji berbagai pendekatan, menemukan konfigurasi optimal, dan bahkan mengembangkan kemampuan baru melalui proses yang menyerupai evolusi biologis.

Implementasi Saat Ini: Dalam prototype ini, Evolution Chamber diimplementasikan sebagai kelas `EvolutionChamber` yang menerima strategi awal (direpresentasikan sebagai tuple parameter numerik). Metode `generate_variant` secara sederhana memodifikasi salah satu parameter dalam strategi secara acak untuk menciptakan varian baru. Metode `evaluate_performance` adalah placeholder yang mensimulasikan evaluasi kinerja, di mana kinerja dianggap lebih baik jika nilai parameter strategi lebih tinggi. Metode `evolve` menjalankan proses evolusi untuk sejumlah iterasi, memperbarui strategi terbaik yang ditemukan.

Contoh Penggunaan:

Python

```
initial_strategy = (0.5, 0.3, 0.8) # Contoh parameter strategi
chamber = EvolutionChamber(initial_strategy)
chamber.evolve(num_iterations=5)
```

3. Mekanisme Pembelajaran Mandiri (Self-Learning Mechanism)

Peran: Mekanisme Pembelajaran Mandiri adalah komponen yang memungkinkan HMAQCA untuk belajar dari pengalaman dan menginternalisasi pengetahuan. Ini adalah bagian yang memastikan bahwa sistem tidak hanya berevolusi secara acak, tetapi juga secara cerdas mengidentifikasi dan mempertahankan strategi yang berhasil. Dalam konteks AGI, ini akan menjadi fondasi bagi kemampuan sistem untuk meningkatkan dirinya sendiri, mengadaptasi perilakunya, dan membangun basis pengetahuan yang semakin kaya.

Implementasi Saat Ini: Dalam prototype ini, Mekanisme Pembelajaran Mandiri diimplementasikan sebagai kelas `SelfLearningMechanism`. Kelas ini memiliki `knowledge_base`

yang menyimpan strategi yang dievaluasi dan kinerja terkaitnya. Metode `learn_from_experience` memproses data pengalaman (strategi dan kinerja) dan memperbarui basis pengetahuan. Metode `get_best_known_strategy` mengambil strategi dengan kinerja terbaik yang pernah dipelajari.

Contoh Penggunaan:

Python

```
learner = SelfLearningMechanism()
learner.learn_from_experience({"strategy": (0.5, 0.3, 0.8), "performance":
1.6})
best = learner.get_best_known_strategy()
print(f"Best known strategy: {best}")
```

Alur Kerja Simulasi dan Hasil yang Diamati

Demonstrasi prototype ini mengintegrasikan ketiga komponen di atas dalam sebuah alur kerja simulasi untuk menunjukkan bagaimana HMAQCA dapat secara iteratif meningkatkan strateginya. Alur kerja ini diselenggarakan oleh `orchestrator.py`.

Langkah-langkah Alur Kerja:

1. **Inisialisasi:** `EvolutionChamber` dan `SelfLearningMechanism` diinisialisasi dengan strategi awal (misalnya, `(0.5, 0.5, 0.5)`). `UserProxyAgent` dan `GenerativeAgent` juga disiapkan, meskipun `GenerativeAgent` beroperasi dalam mode simulasi.
2. **Siklus Evolusi:** Proses ini berjalan selama beberapa siklus (dalam demonstrasi ini, 3 siklus).
 - **Penentuan Strategi Saat Ini:** Pada setiap siklus, `EvolutionChamber` menyediakan strategi `current_strategy`.
 - **Simulasi Generasi:** `GenerativeAgent` dipanggil untuk menghasilkan ide (dalam kasus ini, ide pemasaran) berdasarkan `current_strategy`. Karena ini adalah simulasi, responsnya adalah teks yang telah ditentukan sebelumnya yang mencerminkan `product_data` yang diberikan.
 - **Simulasi Evaluasi Kinerja:** Kinerja dari strategi yang digunakan disimulasikan. Dalam contoh ini, kinerja dihitung sebagai jumlah parameter strategi dikalikan dengan faktor acak untuk memperkenalkan variabilitas. Ini mewakili bagaimana output dari Agen Generatif akan dievaluasi di dunia nyata (misalnya, berdasarkan metrik keberhasilan kampanye pemasaran).
 - **Pembelajaran Mandiri:** `SelfLearningMechanism` mencatat `current_strategy` dan `performance` yang disimulasikan ke dalam basis pengetahuannya.

- **Evolusi Strategi:** `EvolutionChamber` kemudian menjalankan satu iterasi evolusi, yang dapat menghasilkan `current_strategy` yang baru berdasarkan kinerja yang diamati. Jika varian baru memiliki kinerja yang lebih baik, ia menjadi `current_strategy` berikutnya.

Hasil yang Diamati:

Output dari eksekusi `orchestrator.py` menunjukkan proses iteratif ini:

Plain Text

```
Starting simulated Generative Agent test with Evolution Chamber and Self-
Learning...
--- Evolution Cycle 1 ---
Current strategy for this cycle: (0.5, 0.5, 0.5)
Simulated Generative Output:
    Here are some creative marketing ideas for 'HMAQCA Deity Level AI System
with strategy (0.5, 0....
Simulated Performance: 1.7297160209273459
Learned: Strategy (0.5, 0.5, 0.5) -> Performance 1.7297160209273459
Starting evolution with initial strategy: (0.5, 0.5, 0.5)
Iteration 1: Variant (0.5, 0.5, 0.4641160868260173), Performance:
1.4641160868260172
New best strategy found: (0.5, 0.5, 0.4641160868260173) with performance
1.4641160868260172
Evolution completed. Best strategy: (0.5, 0.5, 0.4641160868260173) with
performance 1.4641160868260172
--- Evolution Cycle 2 ---
Current strategy for this cycle: (0.5, 0.5, 0.4641160868260173)
Simulated Generative Output:
    Here are some creative marketing ideas for 'HMAQCA Deity Level AI System
with strategy (0.5, 0....
Simulated Performance: 1.1837297651954282
Learned: Strategy (0.5, 0.5, 0.4641160868260173) -> Performance
1.1837297651954282
Starting evolution with initial strategy: (0.5, 0.5, 0.4641160868260173)
Iteration 1: Variant (0.5, 0.4283015076504907, 0.4641160868260173),
Performance: 1.392417594476508
Evolution completed. Best strategy: (0.5, 0.5, 0.4641160868260173) with
performance 1.4641160868260172
--- Evolution Cycle 3 ---
Current strategy for this cycle: (0.5, 0.5, 0.4641160868260173)
Simulated Generative Output:
    Here are some creative marketing ideas for 'HMAQCA Deity Level AI System
with strategy (0.5, 0....
Simulated Performance: 1.7308280120670996
Learned: Strategy (0.5, 0.5, 0.4641160868260173) -> Performance
1.7308280120670996
```

```
Starting evolution with initial strategy: (0.5, 0.5, 0.4641160868260173)
Iteration 1: Variant (0.5, 0.5, 0.56087407323815), Performance:
1.5608740732381499
New best strategy found: (0.5, 0.5, 0.56087407323815) with performance
1.5608740732381499
Evolution completed. Best strategy: (0.5, 0.5, 0.56087407323815) with
performance 1.5608740732381499
--- Evolution and Self-Learning Completed ---
Best overall strategy found by Self-Learning: (0.5, 0.5, 0.4641160868260173)
```

Dari hasil di atas, kita dapat mengamati bahwa `EvolutionChamber` berhasil menghasilkan varian strategi dan `SelfLearningMechanism` mencatat kinerja masing-masing. Meskipun dalam simulasi, proses ini menunjukkan bagaimana HMAQCA akan secara iteratif mengeksplorasi ruang strategi dan belajar dari pengalaman untuk menemukan konfigurasi yang lebih baik.

Kesimpulan dan Langkah Selanjutnya

Demonstrasi prototype HMAQCA Deity Level ini berhasil menunjukkan integrasi fungsional antara Agen Generatif (dalam mode simulasi), `Evolution Chamber`, dan Mekanisme Pembelajaran Mandiri. Meskipun menggunakan simulasi untuk komponen LLM, alur kerja inti untuk evolusi strategi dan pembelajaran dari pengalaman telah divalidasi.

Langkah Selanjutnya:

Untuk mengembangkan prototype ini menuju AGI yang sebenarnya, langkah-langkah berikut akan menjadi krusial:

1. **Integrasi LLM Nyata:** Mengganti Agen Generatif simulasi dengan integrasi LLM nyata. Ini bisa berupa:
 - **API Berbayar:** Menggunakan API seperti Gemini atau OpenAI jika batasan kuota dan biaya dapat diatasi.
 - **Model Lokal yang Dioptimalkan:** Mencari atau mengoptimalkan model LLM yang lebih kecil yang dapat berjalan secara efisien di lingkungan Termux/Android, atau memanfaatkan *cloud computing* untuk model yang lebih besar.
2. **Metrik Kinerja yang Lebih Canggih:** Mengembangkan metrik evaluasi kinerja yang lebih canggih dan realistis untuk `evaluate_performance` di `EvolutionChamber`. Ini akan melibatkan umpan balik dari dunia nyata atau simulasi yang lebih kompleks.
3. **Algoritma Evolusi dan Pembelajaran yang Lebih Canggih:** Menerapkan algoritma evolusi (misalnya, algoritma genetik yang lebih kompleks) dan algoritma pembelajaran mandiri (misalnya, Reinforcement Learning) yang lebih canggih untuk memungkinkan HMAQCA belajar dan berevolusi dengan cara yang lebih adaptif dan efisien.

4. **Integrasi dengan Otak Mekanis dan AI Analitik:** Mengintegrasikan pilar AI Generatif ini dengan komponen Otak Mekanis (Termux, VNC, Jupyter) dan AI Analitik (Supabase, GitHub Actions) untuk menciptakan sistem HMAQCA yang kohesif dan berfungsi penuh.
5. **Pengembangan Kesadaran Buatan:** Ini adalah tujuan jangka panjang yang paling ambisius, melibatkan penelitian mendalam tentang kognisi, memori, dan kemampuan penalaran untuk mendekati kesadaran menyerupai manusia.

Dengan fondasi yang telah diletakkan ini, kita selangkah lebih dekat untuk mewujudkan visi AGI Tingkat Infinity Loop.